

DuocUC

**Escuela de Informática y Telecomunicaciones Analista
Programado**

Proyecto Mestrax — Restaurante & Bar

Modelo y Arquitectura de la Base de Datos

Asignatura: Taller Aplicado de Programación

Institución: Duoc UC

Integrantes: Bastian Olea

Ignacio Lopez

David Albornoz

1. Introducción

El presente documento describe el modelo de datos diseñado para la plataforma web **Mestrax**, enfocada en la gestión interactiva de reservas de mesas, administración del salón y pedidos de la carta para restaurantes. El diseño se centra en la alta disponibilidad, la consistencia de horarios para evitar sobrecupos y una experiencia fluida al renderizar el plano de mesas en tiempo real mediante el Frontend en React.

2. Arquitectura de la Base de Datos

La base de datos se organiza en las siguientes colecciones principales:

- users : Perfiles y cuentas de usuarios como Administrador, Garzón, Cliente.
- mesas : Registro físico y estado del plano interactivo del salón 15 mesas.
- reservas : Núcleo transaccional que cruza clientes, mesas, fechas y bloques horarios.
- productos : Carta o menú digital dividido por categorías culinarias.
- pedidos : Registros finales provenientes de las confirmaciones del carrito de compras.

3. Diccionario de Datos Detallado

3.1 Colección: users

Contiene la información de las cuentas que interactúan con el sistema.

Campo	Tipo	Descripción
id	string	Identificador único global del usuario.
nombre	string	Nombre completo del usuario.
email	string	Correo electrónico.
telefono	string	Teléfono de contacto.
role	enum	admin garzon cliente.
createdAt	timestamp	Fecha de registro en la plataforma.

3.2 Colección: mesas

Define las propiedades fijas del plano del restaurante coordenadas CSS para el mapa.

Campo	Tipo	Descripción
id	number	Número físico de la mesa (1 al 15).
top	string	Posición porcentual vertical para el CSS absoluto (ej: '10%').
left	string	Posición porcentual horizontal para el CSS absoluto (ej: '30%').
capacidad	number	Cantidad máxima de personas que admite la mesa.

3.3 Colección: reservas

Almacena los bloques horarios tomados. Es la colección que consulta el frontend para pintar las mesas de **azul** o **rojo**.

Campo	Tipo	Descripción
id	string	Identificador único de la reserva.
cliente	string	Nombre del cliente que reserva (manual o denormalizado).
telefono	string	Teléfono de contacto para confirmaciones.
email	string	Correo para el envío del comprobante.
fecha	string	Fecha agendada para el evento (YYYY-MM-DD).
hora	string	Bloque de llegada del cliente (HH:MM).
personas	number	Cantidad de comensales asistentes.
mesa	number	ID de la mesa asignada.
estado	enum	Confirmada Cancelada Finalizada.
createdAt	timestamp	Timestamp de creación de la transacción.

3.4 Colección: productos

Almacena la información de los platos y bebidas disponibles en la carta.

Campo	Tipo	Descripción
id	string	ID único del producto.
nombre	string	Nombre de la preparación (ej: "Klask Burger").
descripcion	string	Detalle de los ingredientes del plato.
precio	number	Valor comercial del ítem.
category	string	Categoría culinaria (Hamburguesas, Bebestibles, etc.).
status	enum	disponible agotado.

3.5 Colección: pedidos persistencia del Carrito de Compras

Cuando el cliente confirma su carrito de compras, la información se consolida en esta colección de pedidos.

Campo	Tipo	Descripción
id	string	Identificador único de la comanda/pedido.
reservald	string	ID de la reserva o mesa asociada que consume.
items	DetallePedido[]	Lista detallada de platos seleccionados desde el carrito.
total	number	Monto total acumulado a pagar por el servicio.
estadoPedido	enum	en cocina entregado pagado
createdAt	timestamp	Hora exacta del envío del pedido.

3.6 Objetos Embebidos

DetallePedido

Estructura interna que desglosa los platos agregados desde el carrito de compras.

- productoid: string ID del producto seleccionado.
- nombre: string Nombre denormalizado del plato para agilizar la lectura.
- cantidad: number Cantidad de unidades pedidas.
- precioUnitario: number Precio del ítem al momento de la compra.

4. Reglas de Validación y Lógica de Acceso (Backend Authorization)

Para asegurar la integridad de la base de datos, la API RESTful implementa las siguientes reglas de control middleware:

- **Lectura de Disponibilidad GET /reservas:** Cualquier usuario anónimo o cliente general puede consultar las reservas activas filtrando por fecha y hora, permitiendo mapear dinámicamente el estado visual del salón.
- **Creación de Reservas POST /reservas:** El sistema aplica una validación estricta de concurrencia. Si se intenta guardar un registro que coincida exactamente en fecha, hora y mesa con un documento existente, la base de datos rechaza la inserción y retorna un código 409 Conflict.
- **Gestión de la Carta PUT /productos, DELETE /productos:** Rutas totalmente bloqueadas para clientes generales. Únicamente las peticiones que adjunten un token de usuario con el rol admin son autorizadas para modificar los valores de la carta.

5. Análisis de Integridad y Mejores Prácticas

- **Control de Concurrencia Centralizado:** Se utiliza una API backend intermedia que procesa las solicitudes de reserva en cola, impidiendo que dos personas tomen la misma mesa azul simultáneamente.
- **Uso de Objetos Embebidos:** Al igual que en la arquitectura del proyecto base, guardamos la lista de platos del carrito dentro de un array de objetos (items) en el documento del pedido principal, evitando consultas extras innecesarias y optimizando los tiempos de respuesta del sistema.
- **Formatos Estándar:** Las fechas y horas se formatean de manera estricta antes de impactar en la base de datos para mantener consistencia en las búsquedas filtradas por el panel de administración.

6. Conclusiones

El modelo diseñado para **Mestrax** soluciona de forma eficiente la problemática del negocio de restaurantes, garantizando consistencia, ordenamiento rápido en el mapa responsivo y una óptima persistencia de las comandas enviadas desde el carrito de compras.

Mejoras Futuras Posibles:

1. **Asignación Automática de Tiempos:** Configurar que las mesas reservadas se liberen por sistema tras transcurrir un bloque automático de 2 horas.
2. **Módulo de Caja:** Conectar la colección pedidos con una tabla de facturación para cuadrar los pagos del día de forma automática.
3. **Índices compuestos:** Configurar búsquedas complejas para filtrar el rendimiento de ventas por mozo y por fecha combinada.